

Stock Earnings Agent

Technical Study Guide (Line by Line)

Purpose: to help you understand and explain every part of this project end to end, at the depth needed for an interview. It starts with the underlying concepts, then walks the repository file by file, then traces complete requests through the code, and finishes with likely interview questions.

How to read this guide. Read Part 1 once to build vocabulary. After that, each file section is self contained. Code is shown in blocks, followed by explanations of the important lines, the syntax used, and the reasoning (the why, the how, and the what).

Part 1. Concepts and basics you need first

These ideas appear throughout the code. Understand them once and the rest reads easily.

1.1 Modules, packages and `__init__.py`

A single `.py` file is a **module**. A folder that contains an `__init__.py` file is a **package** (a group of modules you can import as one name). When you write `from core import get_settings`, Python runs `core/__init__.py` and looks for the name `get_settings` there. In this project each package's `__init__.py` re-exports the useful names so callers can write `from rag_pipeline import RAGPipeline` instead of the longer `from rag_pipeline.pipeline import RAGPipeline`.

1.2 `from __future__ import annotations`

This line appears at the top of most files. It makes Python treat all type hints as plain text (it does not evaluate them at runtime). Two benefits: you can use modern syntax such as `str | None` on older versions, and you avoid import cycles because a type used only in a hint does not need to be imported for real. It has no effect on how the program runs, only on how annotations are stored.

1.3 Type hints

`def f(x: int) -> str:` says `x` should be an integer and the function returns a string. Python does not enforce this at runtime; hints are for humans, editors and tools. Common hints in this repo:

- `str | None` means "a string or nothing" (`None`).
- `list[float]` means a list of floats. `list[list[float]]` is a list of such lists (a matrix), which is how a batch of embeddings is represented.
- `dict[str, Any]` means a dictionary with string keys and values of any type.
- `Sequence[str]` means "any ordered collection of strings" (list, tuple, and so on). It is more general than `list[str]`.

1.4 Docstrings

The triple quoted string just under a `def`, `class` or at the top of a file is a **docstring**. It documents what the thing does. Tools and editors show it as help. It is not a comment; it is stored on the object as `.__doc__`.

1.5 Decorators

A decorator is a function that wraps another function or class to add behaviour, written with an `@` above the definition. Examples used here:

- `@dataclass` generates boilerplate for a class that mainly holds data.
- `@lru_cache` remembers results so repeated calls are instant.
- `@property` exposes a method as if it were an attribute.
- `@mcp.tool()` registers a function as a callable tool on the MCP server.
- `@tool` (LangChain) turns a function into a tool the agent can call.
- `@retry(...)` re-runs a function automatically when it fails.

1.6 Dataclasses

`@dataclass` writes the repetitive parts of a data holding class for you (the constructor, equality, a readable representation). `@dataclass(slots=True)` also tells Python to store fields in a fixed, memory efficient layout instead of a per instance dictionary.

1.7 Pydantic and settings

Pydantic validates data against a typed model. If a value has the wrong type or breaks a rule, it raises an error immediately with a clear message. **pydantic-settings** extends this to read values from environment variables and a `.env` file. This project uses it so that all configuration is checked once at startup rather than failing deep inside a request.

1.8 Caching with `lru_cache`

`@lru_cache` stores the return value of a function keyed by its arguments. The next call with the same arguments returns the stored value without running the body. With `maxsize=1` and no arguments it becomes a simple "run once" cache, which is how settings are parsed a single time per process.

1.9 Context managers (the `with` statement)

`with open(path) as f:` guarantees the file is closed when the block ends, even if an error happens. Any object that supports this protocol can be used with `with`. The code uses it for files and for opening PDFs.

1.10 Generators, `yield` and `yield from`

A function that uses `yield` is a **generator**: it produces values one at a time instead of building a whole list. This is useful for streaming. `yield from iterable` forwards every value from another generator, which is a concise way to re-stream results.

1.11 Exceptions and the "ask forgiveness" style

Python commonly tries an operation inside `try` and handles failure in `except`, rather than checking everything in advance. This is called EAFP (easier to ask forgiveness than permission). The tool functions use it to catch network or data errors and return a structured error value instead of crashing.

1.12 f-strings

`f"total is {value}"` builds a string with `value` inserted. Anything in braces is evaluated. `f"{x:.2f}"` formats a number to two decimal places.

1.13 `pathlib.Path`

`Path` is the modern way to handle file paths. `Path(__file__)` is the current file; `.resolve()` makes it absolute; `.parent` goes up one folder; the `/` operator joins paths (`Path("a") / "b"`). Using `Path` instead of raw strings makes the code work the same on Windows and Linux.

1.14 REST APIs and HTTP

A REST API is called over HTTP. You send a request to a URL (often with a JSON body) and receive a response with a status code (200 means success, 4xx means the caller made a mistake, 5xx means the server failed) and usually a JSON body. This project calls Google for embeddings and the SEC for filings this way, using the `requests` and `httpx` libraries.

1.15 Embeddings, vectors and cosine similarity

An **embedding** is a list of numbers (a vector) that represents the meaning of a piece of text. Texts with similar meaning have vectors that point in similar directions. **Cosine similarity** measures the angle between two vectors: 1 means identical direction, 0 means unrelated. Search works by embedding the query and finding stored vectors with the highest cosine similarity.

1.16 RAG (retrieval augmented generation)

RAG means: before asking the language model to answer, retrieve relevant text from your own documents and give it to the model as context. This grounds the answer in real source material and lets the model cite it. The pipeline here does the retrieval half; the agent uses it as one of its tools.

1.17 Tool calling and the ReAct agent

A modern language model can be given a list of **tools** (functions with descriptions). Instead of guessing an answer, it can decide to call a tool, read the result, and continue. **ReAct** is the loop of Reason then Act: think, call a tool, observe the result, repeat, then answer. **LangGraph** builds and runs this loop as a graph.

1.18 MCP (Model Context Protocol)

MCP is a standard that lets an AI client (such as Claude) discover and call tools that a server exposes. The server here publishes six tools over MCP. The important practical detail is that MCP talks over

standard output, so the program must never print logs to standard output; logs go to standard error instead.

Part 2. Repository structure

```
stock-earnings-agent/
  core/           configuration and logging (shared foundation)
  rag_pipeline/   ingest, chunk, embed, store, retrieve
  mcp_server/     MCP server + shared tool implementations
  agents/         the LangGraph reasoning agent
  app/           the Streamlit web application
  evaluation/     retrieval and RAGAS quality metrics
  scripts/        command line utilities (ingest, healthcheck, seed)
  tests/          unit and integration tests
  pyproject.toml  project metadata, dependencies, tool settings
  requirements.txt runtime dependencies
  Dockerfile      container image definition
  Makefile        common developer commands
```

The dependency direction is one way: everything can use `core`; `agents` and `mcp_server` use `rag_pipeline` and the shared tools; `app` uses everything. There are no cycles.

Part 3. File by file

3.1 `core/config.py`

This file defines every configurable value as a typed, validated object.

```
from __future__ import annotations
from functools import lru_cache
from pathlib import Path
from pydantic import Field, field_validator
from pydantic_settings import BaseSettings, SettingsConfigDict

PROJECT_ROOT: Path = Path(__file__).resolve().parent.parent
```

- The imports bring in caching, path handling, and the Pydantic tools.
- `PROJECT_ROOT` computes the repository root once. `__file__` is this file's path, `.resolve()` makes it absolute, and `.parent.parent` climbs from `core/config.py` up to the project folder.

Doing this relative to the file means the app finds the `.env` file no matter which directory it is launched from.

```
class Settings(BaseSettings):
    model_config = SettingsConfigDict(
        env_file=PROJECT_ROOT / ".env",
        env_file_encoding="utf-8",
        case_sensitive=False,
        extra="ignore",
    )
```

- `Settings` inherits from `BaseSettings`, which knows how to load fields from the environment.
- `model_config` configures that loading: read the project `.env`, treat env names case-insensitively (so `GROQ_API_KEY` fills the field `groq_api_key`), and ignore any unrelated variables instead of failing.

```
groq_api_key: str = Field(..., description="API key for Groq inference.")
google_api_key: str = Field(
    ..., description="API key for Google Generative Language (embeddings)."
```

- Each line declares a field, its type, and a default via `Field`. The literal `...` (Ellipsis) means **required**: if it is missing, validation fails. These two are the only required values, so the app cannot start without its keys.

```
groq_model: str = Field("qwen/qwen3.8-27b", description="...")
llm_temperature: float = Field(0.1, ge=0.0, le=2.0)
llm_max_tokens: int = Field(800, gt=0)
```

- These have defaults, so they are optional. `Field` also carries validation rules: `ge=0.0, le=2.0` means "greater or equal to 0 and less or equal to 2"; `gt=0` means "greater than 0". `temperature` controls randomness of the model (low is focused). `max_tokens` caps the answer length; it is kept modest to stay within the hosted tier's per request limit.
- The remaining fields (embedding model and dimension, Chroma directory and collection, chunk size and overlap, retrieval count, HTTP timeout, SEC user agent, log level) follow the same pattern: a name, a type, a default, sometimes a rule.

```

@field_validator("log_level")
@classmethod
def _normalise_log_level(cls, value: str) -> str:
    level = value.upper()
    valid = {"TRACE", "DEBUG", "INFO", "SUCCESS", "WARNING", "ERROR", "CRITICAL"}
    if level not in valid:
        raise ValueError(f"log_level must be one of {sorted(valid)}")
    return level

```

- `@field_validator("log_level")` runs custom checking for that one field. `@classmethod` means it receives the class, not an instance. It upper cases the value, checks it against the allowed set (a `set`, which gives fast membership tests), raises a clear error if invalid, and returns the cleaned value that gets stored. This is the "why": fail early with a helpful message.

```

@field_validator("chunk_overlap")
@classmethod
def _overlap_below_size(cls, value: int, info) -> int:
    size = info.data.get("chunk_size")
    if size is not None and value >= size:
        raise ValueError("chunk_overlap must be smaller than chunk_size")
    return value

```

- This validator depends on another field. `info.data` holds the fields validated so far, so it can compare overlap against size. An overlap larger than the chunk would be nonsensical, so it is rejected.

```

@property
def embedding_endpoint(self) -> str:
    return (
        "https://generativelanguage.googleapis.com/v1/models/"
        f"{self.embedding_model}:embedContent"
    )

```

- `@property` turns a method into a computed read only attribute, used as `settings.embedding_endpoint`. It builds the exact URL from the configured model name. The two adjacent strings are automatically joined; the second is an f-string that inserts the model name.

```

@lru_cache(maxsize=1)
def get_settings() -> Settings:
    return Settings() # type: ignore[call-arg]

```

- `get_settings` is the single entry point the whole app uses. `@lru_cache(1)` means the `Settings()` object is built once and reused, so the `.env` file is parsed a single time. `# type: ignore[call-arg]` silences a false type warning (the required fields are filled from the environment, which the type checker cannot see). If a required key is missing this call raises, which is the intended loud failure at startup.

3.2 `core/logging.py`

Centralised logging built on Loguru, deliberately writing to standard error.

```
import os, sys
from loguru import logger
_CONFIGURED = False
_LOG_FORMAT = "... {time} | {level} | {name}:{function}:{line} - {message} ..."
```

- `logger` is Loguru's ready made logger. `_CONFIGURED` is a module level flag so setup runs only once. `_LOG_FORMAT` defines each log line's shape.

```
def setup_logging(level: str | None = None) -> None:
    global _CONFIGURED
    resolved = (level or os.getenv("LOG_LEVEL", "INFO")).upper()
    logger.remove()
    logger.add(sys.stderr, level=resolved, format=_LOG_FORMAT, ...)
    _CONFIGURED = True
```

- `global _CONFIGURED` lets the function update the module flag.
- `level or os.getenv(...)` picks the passed level, else the environment value, else `INFO`. It reads the environment directly rather than through settings so logging works even before settings are validated.
- `logger.remove()` clears Loguru's default handler; `logger.add(sys.stderr, ...)` sends logs to standard error. This is the key line for MCP safety: standard output stays clean for the protocol.

```
def get_logger(name: str | None = None):
    if not _CONFIGURED:
        setup_logging()
    return logger.bind(component=name) if name else logger
```

- Any module calls `get_logger("something")`. On first use it configures logging. `logger.bind(component=name)` tags every message from that caller with a component name, which makes logs easier to filter.

3.3 rag_pipeline/ingestion.py

Turns files on disk into `Document` objects with metadata.

```
from collections.abc import Iterable
from dataclasses import dataclass, field
from pathlib import Path
from typing import Any
import pdfplumber
from core import get_logger
log = get_logger("ingestion")
SUPPORTED_SUFFIXES = {".pdf", ".txt", ".md", ".markdown"}
```

- `pdfplumber` reads PDFs in pure Python. `SUPPORTED_SUFFIXES` is a set of allowed file extensions; membership checks against a set are fast.

```
@dataclass(slots=True)
class Document:
    content: str
    metadata: dict[str, Any] = field(default_factory=dict)
```

- A small data holder: the text and its metadata. `field(default_factory=dict)` means each `Document` gets its **own** empty dictionary. You must use a factory here; a plain `= {}` default would be shared by all instances, a classic Python trap. `slots=True` makes instances lighter.

```
def load_path(path: str | Path, metadata=None) -> list[Document]:
    path = Path(path)
    if not path.exists():
        raise FileNotFoundError(f"Document not found: {path}")
    suffix = path.suffix.lower()
    if suffix not in SUPPORTED_SUFFIXES:
        raise ValueError(f"Unsupported file type '{suffix}'. ...")
    base_meta = {"source": path.name, **(metadata or {})}
```

- Accepts a string or a `Path` and normalises to `Path`. It checks existence and file type up front, raising specific exceptions for each problem.
- `base_meta` records the file name as `source` and merges any caller supplied metadata. `{**a, **b}` builds a new dict by unpacking both; `metadata or {}` guards against `None`.

```

if suffix == ".pdf":
    docs = _load_pdf(path, base_meta)
else:
    docs = _load_text(path, base_meta)
log.info("Loaded {} document section(s) from {}", len(docs), path.name)
return docs

```

- It dispatches to a PDF or text loader and logs how much it read. Loguru uses `{}` placeholders filled by the extra arguments, which is cheaper than building the string when the log level would hide it.

```

def load_documents(paths, metadata=None) -> list[Document]:
    documents = []
    for path in paths:
        try:
            documents.extend(load_path(path, metadata))
        except (FileNotFoundError, ValueError, OSError) as exc:
            log.warning("Skipping {}: {}", path, exc)
    return documents

```

- Bulk loader. It catches per file errors and skips the bad file with a warning, so one unreadable file does not abort a whole batch. `extend` adds all items from a list; `append` would add the list itself.

```

def _load_pdf(path, base_meta):
    documents = []
    with pdfplumber.open(path) as pdf:
        for page_number, page in enumerate(pdf.pages, start=1):
            text = (page.extract_text() or "").strip()
            if not text:
                continue
            documents.append(Document(content=text, metadata={**base_meta,
                "page": page_number, "doc_type": base_meta.get("doc_type", "pdf")}))
    ...

```

- The leading underscore in `_load_pdf` signals "internal, not part of the public API". The `with` block guarantees the PDF is closed. `enumerate(..., start=1)` gives human friendly page numbers. `page.extract_text() or ""` handles pages that return `None`. Blank pages are skipped with `continue`. Each page becomes its own `Document`, carrying its page number so later citations can point to it.

`_load_text` reads a whole text or markdown file with UTF-8 and `errors="replace"` so odd bytes never crash the read.

3.4 `rag_pipeline/chunking.py`

Splits long documents into overlapping pieces sized for embedding.

```
from langchain_text_splitters import RecursiveCharacterTextSplitter

def _build_splitter(chunk_size, chunk_overlap):
    return RecursiveCharacterTextSplitter(
        chunk_size=chunk_size, chunk_overlap=chunk_overlap,
        separators=["\n\n", "\n", ". ", " ", ""], length_function=len)
```

- The splitter tries to break text on the most natural boundary available, in order: paragraph, line, sentence, word, and finally any character. `chunk_size` is the target length; `chunk_overlap` repeats some text between neighbours so an idea split across a boundary is not lost. `length_function=len` measures size in characters.

```
def chunk_documents(documents, chunk_size=None, chunk_overlap=None):
    settings = get_settings()
    splitter = _build_splitter(chunk_size or settings.chunk_size,
                               chunk_overlap if chunk_overlap is not None else
    settings.chunk_overlap)
    chunks = []
    for document in documents:
        pieces = splitter.split_text(document.content)
        for index, piece in enumerate(pieces):
            chunks.append(Document(content=piece, metadata={**document.metadata, "chunk":
index}))
    return chunks
```

- Defaults come from settings when the caller does not override them. Note the overlap check uses `is not None` rather than `or`, because `0` is a valid overlap and `0 or default` would wrongly pick the default.
- Each output chunk inherits the parent document's metadata and adds its own `chunk` index, so every stored piece remains traceable to its source and position.

3.5 `rag_pipeline/embeddings.py`

A resilient client that turns text into vectors by calling Google's REST API.

```

from collections.abc import Sequence
import requests
from tenacity import retry, retry_if_exception_type, stop_after_attempt, wait_exponential
from core import get_logger, get_settings
TASK_RETRIEVAL_DOCUMENT = "RETRIEVAL_DOCUMENT"
TASK_RETRIEVAL_QUERY = "RETRIEVAL_QUERY"

class EmbeddingError(RuntimeError):
    """Raised when the embedding API returns an unrecoverable error."""

```

- `tenacity` provides automatic retries. The two task constants tell Gemini whether it is embedding a stored document or a search query; it tunes the vector accordingly. `EmbeddingError` is a custom exception type so callers can catch this specific failure.

```

class GoogleEmbeddingClient:
    def __init__(self, api_key=None, model=None, timeout=None, batch_size=None):
        settings = get_settings()
        self._api_key = api_key or settings.google_api_key
        self._model = model or settings.embedding_model
        ...
        self._session = requests.Session()

```

- The constructor pulls defaults from settings but allows overrides (useful in tests). A `requests.Session` reuses network connections across calls, which is faster than a fresh connection each time. The leading underscores mark these as internal attributes.

```

def embed_query(self, text: str) -> list[float]:
    return self._embed_one(text, TASK_RETRIEVAL_QUERY)

def embed_documents(self, texts: Sequence[str]) -> list[list[float]]:
    cleaned = [t for t in texts]
    if not cleaned:
        return []
    vectors = []
    for start in range(0, len(cleaned), self._batch_size):
        batch = cleaned[start:start + self._batch_size]
        vectors.extend(self._embed_batch(batch, TASK_RETRIEVAL_DOCUMENT))
    return vectors

```

- `embed_query` embeds one search string. `embed_documents` embeds many, splitting them into batches of `batch_size` so a single request is never too large. The `range(0, n, step)` pattern

walks the list in windows; slicing `list[a:b]` takes a sub list. The method returns a list of vectors in the same order as the input.

```
@retry(
    retry=retry_if_exception_type((requests.RequestException, EmbeddingError)),
    stop=stop_after_attempt(4),
    wait=wait_exponential(multiplier=1, min=1, max=10),
    reraise=True,
)

def _embed_one(self, text, task_type):
    payload = {"model": f"models/{self._model}",
               "content": {"parts": [{"text": text}], "taskType": task_type}
    response = self._session.post(self._single_url, params={"key": self._api_key},
                                  json=payload, timeout=self._timeout)

    self._raise_for_status(response)
    data = response.json()
    try:
        return data["embedding"]["values"]
    except (KeyError, TypeError) as exc:
        raise EmbeddingError(f"Unexpected embedding response: {data}") from exc
```

- The `@retry` decorator re-runs the method on network errors or `EmbeddingError`, up to four attempts, waiting an increasing amount between tries (1s, 2s, 4s, capped at 10s). This handles transient blips without giving up. `reraise=True` means the real error surfaces after the final attempt.
- The `payload` is the JSON body the API expects. `session.post(...)` sends it; the key is passed as a query parameter, and `timeout` prevents hanging forever.
- `data["embedding"]["values"]` reads the vector out of the response. If the shape is unexpected, it raises `EmbeddingError` with `from exc` to preserve the original cause for debugging.

`_embed_batch` is the same idea but posts a list of requests and returns a list of vectors.

`_raise_for_status` inspects the HTTP status code: 200 passes, 5xx and 429 (rate limited) raise a retryable `EmbeddingError`, and other 4xx errors raise immediately because retrying a bad request will not help.

3.6 rag_pipeline/vector_store.py

Wraps ChromaDB as a persistent similarity index.

```

_CLIENT_CACHE: dict[str, chromadb.api.ClientAPI] = {}

def _get_client(persist_dir: Path) -> chromadb.api.ClientAPI:
    key = str(persist_dir.resolve())
    client = _CLIENT_CACHE.get(key)
    if client is None:
        persist_dir.mkdir(parents=True, exist_ok=True)
        client = chromadb.PersistentClient(path=key,
            settings=ChromaSettings(anonymized_telemetry=False, allow_reset=True))
        _CLIENT_CACHE[key] = client
    return client

```

- This is an important design detail. Chroma's native core must have only one client per database path in a process; creating a second can crash the program. The module level dictionary caches clients by absolute path, so every part of the app shares one client. `mkdir(parents=True, exist_ok=True)` creates the folder if needed and does not error if it already exists. Telemetry is turned off.

```

def _stable_id(text, metadata) -> str:
    source = str(metadata.get("source", ""))
    chunk = str(metadata.get("chunk", ""))
    page = str(metadata.get("page", ""))
    digest = hashlib.sha256(f"{source}|{page}|{chunk}|{text}".encode()).hexdigest()
    return digest[:32]

```

- Each chunk gets a deterministic id derived from its source, page, chunk index and text using a SHA-256 hash. "Deterministic" means the same content always produces the same id, so re-ingesting a file updates existing rows instead of creating duplicates. `.encode()` turns the string into bytes (hashing works on bytes); `[:32]` keeps the first 32 hex characters as a compact id.

```

class ChromaVectorStore:
    def __init__(self, persist_dir=None, collection_name=None):
        settings = get_settings()
        self._persist_dir = Path(persist_dir or settings.chroma_persist_dir)
        self._collection_name = collection_name or settings.chroma_collection
        self._client = _get_client(self._persist_dir)
        self._collection = self._client.get_or_create_collection(
            name=self._collection_name, metadata={"hnsw:space": "cosine"})

```

- A **collection** is like a table of vectors. `get_or_create_collection` opens it if present or creates it. `{"hnsw:space": "cosine"}` tells Chroma to rank results by cosine similarity, matching how

the embeddings are meant to be compared. HNSW is the fast approximate nearest neighbour index Chroma uses under the hood.

```
def add(self, texts, embeddings, metadatas) -> list[str]:
    if not (len(texts) == len(embeddings) == len(metadatas)):
        raise ValueError("texts, embeddings and metadatas must be the same length")
    if not texts:
        return []
    ids = [_stable_id(t, m) for t, m in zip(texts, metadatas, strict=True)]
    self._collection.upsert(ids=ids, documents=list(texts),
        embeddings=[list(e) for e in embeddings], metadatas=list(metadatas))
    return ids
```

- It first checks the three inputs line up, failing loudly otherwise. The chained comparison `a == b == c` reads naturally in Python.
- `zip(texts, metadatas, strict=True)` pairs each text with its metadata; `strict` raises if the lengths differ (a safety net). The list comprehension builds one id per pair.
- `upsert` means insert or update: existing ids are overwritten, new ids are added. This is what makes re-ingestion safe.

```
def query(self, query_embedding, top_k=None, where=None) -> list[dict]:
    settings = get_settings()
    k = top_k or settings.retrieval_top_k
    if self.count() == 0:
        return []
    result = self._collection.query(query_embeddings=[list(query_embedding)],
        n_results=min(k, self.count()), where=where or None,
        include=["documents", "metadatas", "distances"])
    documents = result.get("documents", [[]])[0]
    metadatas = result.get("metadatas", [[]])[0]
    distances = result.get("distances", [[]])[0]
    hits = []
    for text, metadata, distance in zip(documents, metadatas, distances, strict=False):
        hits.append({"text": text, "metadata": metadata or {},
            "score": round(1.0 - float(distance), 4)})
    return hits
```

- It guards the empty store case, then asks Chroma for the nearest `k` vectors. `n_results=min(k, count)` avoids asking for more than exist. `where` applies an optional metadata filter such as `{"ticker": "AAPL"}`.
- Chroma returns lists nested one level deep (one entry per query); `[0]` takes the first query's results. Chroma reports **distance**; the code converts it to a **similarity** with `1.0 - distance`

and rounds it, so a higher score means more relevant, which is easier to reason about.

`reset` deletes and recreates the collection and is only for maintenance.

3.7 `rag_pipeline/pipeline.py`

Ties the pieces together behind two verbs: ingest and retrieve.

```
@dataclass(slots=True)
class RetrievedChunk:
    text: str
    metadata: dict[str, Any]
    score: float

    @property
    def citation(self) -> str:
        source = self.metadata.get("source", "unknown")
        page = self.metadata.get("page")
        return f"{source} (p.{page})" if page else str(source)
```

- A typed result for retrieval. The `citation` property builds a readable source label, adding a page number when one exists, using a conditional expression (`value_if_true if condition else value_if_false`).

```
class RAGPipeline:
    def __init__(self, vector_store=None, embedding_client=None):
        self._store = vector_store or ChromaVectorStore()
        self._embedder = embedding_client or GoogleEmbeddingClient()
```

- The pipeline accepts its store and embedder as arguments but creates real ones by default. This is **dependency injection**: tests can pass fakes, production uses the defaults. It is why the code is easy to test without network access.

```
def ingest_documents(self, documents) -> int:
    if not documents:
        return 0

    chunks = chunk_documents(documents)
    if not chunks:
        return 0

    texts = [c.content for c in chunks]
    metadatas = [c.metadata for c in chunks]
    embeddings = self._embedder.embed_documents(texts)
    self._store.add(texts=texts, embeddings=embeddings, metadatas=metadatas)
    return len(chunks)
```

- The write path: chunk the documents, pull out the texts and metadata into parallel lists, embed the texts in one batched call, and store everything. It returns how many chunks were written. The parallel list pattern keeps text, vector and metadata aligned by position.

```
def retrieve(self, query, top_k=None, ticker=None) -> list[RetrievedChunk]:
    settings = get_settings()
    k = top_k or settings.retrieval_top_k
    where = {"ticker": ticker.upper()} if ticker else None
    query_embedding = self._embedder.embed_query(query)
    hits = self._store.query(query_embedding, top_k=k, where=where)
    return [RetrievedChunk(text=h["text"], metadata=h["metadata"], score=h["score"]) for h
in hits]
```

- The read path: build an optional ticker filter, embed the query, search the store, and convert the raw dicts into typed `RetrievedChunk` objects. Embedding the query with the query task type (inside the client) matches it against the documents that were embedded with the document task type.

`format_context` joins the retrieved chunks into one text block with numbered citations, ready to hand to the language model.

3.8 `mcp_server/tools/financials.py`

The market data tools. All six shared tools follow the same contract: validate input, do the work inside `try`, and return a dict, using `{"error": ...}` for expected failures rather than raising.

```
def _clean_ticker(ticker: str) -> str:
    cleaned = (ticker or "").strip().upper()
    if not cleaned or not cleaned.replace(".", "").replace("-", "").isalnum():
        raise ValueError(f"Invalid ticker symbol: {ticker!r}")
    return cleaned
```

- Normalises and validates a ticker. `strip()` removes surrounding spaces, `upper()` standardises case. The check strips out allowed punctuation (`.` and `-`) and then requires the rest to be letters or digits, blocking junk input. `{ticker!r}` inserts the value using its `repr`, which quotes strings so the error is unambiguous.

```
def get_financials(ticker: str) -> dict[str, Any]:
    try:
        symbol = _clean_ticker(ticker)
        info = yf.Ticker(symbol).info
    except ValueError as exc:
        return {"error": str(exc)}
    except Exception as exc:
        log.warning("get_financials({}) failed: {}".format(ticker, exc))
        return {"error": f"Failed to fetch financials for {ticker}: {exc}"}
    if not info or (info.get("longName") is None and info.get("shortName") is None):
        return {"error": f"No financial data found for {ticker}"}
    return {"ticker": symbol, "company_name": info.get("longName") or info.get("shortName"),
        ...}
```

- `yf.Ticker(symbol).info` calls the market data library and returns a big dictionary. Two `except` blocks separate a bad ticker (a `ValueError` from the validator) from any other failure (network, library). The catch all logs and returns a friendly error. `info.get("key")` reads a value or `None` if missing, which never raises even when a field is absent. The final dict maps the library's field names to clean, consistent output keys.

The other functions in this file (`get_price_history`, `calculate_ratios`, `get_analyst_recommendations`) repeat this shape. `get_price_history` also validates the period against an allowed set and computes the percentage change and average volume from the returned price table. `calculate_ratios` reads valuation ratios, with `info.get("pegRatio")` or `info.get("trailingPegRatio")` to handle a field the provider renamed. `get_analyst_recommendations` reads the consensus and, inside its own small `try`, adds the most recent rating changes if available.

3.9 `mcp_server/tools/filings.py`

Fetches official filings from the SEC in three network steps.

```

@lru_cache(maxsize=1)
def _ticker_to_cik_map() -> dict[str, str]:
    settings = get_settings()
    headers = {"User-Agent": settings.sec_user_agent}
    response = httpx.get(_TICKERS_URL, headers=headers, timeout=settings.http_timeout)
    response.raise_for_status()
    mapping = {}
    for entry in response.json().values():
        mapping[entry["ticker"].upper()] = str(entry["cik_str"]).zfill(10)
    return mapping

```

- The SEC identifies companies by a number called a CIK, not by ticker. This function downloads the full ticker to CIK table once (`@lru_cache`) and builds a lookup dictionary. The SEC requires a descriptive `User-Agent` header, which is set from settings. `zfill(10)` pads the number with leading zeros to ten digits, the format the next endpoint expects. `raise_for_status()` turns a bad HTTP status into an exception.

```

def fetch_sec_filing(ticker, form_type="10-Q") -> dict[str, Any]:
    symbol = (ticker or "").strip().upper()
    if not symbol:
        return {"error": "Ticker symbol is required"}
    form = form_type.strip().upper()
    if form not in _VALID_FORMS:
        return {"error": f"Unsupported form_type {form_type!r}. ..."}
    ...
    cik = _ticker_to_cik_map().get(symbol)
    if not cik:
        return {"error": f"CIK not found for ticker {symbol}"}

```

- It validates the ticker and the form type (only `10-Q`, `10-K`, `8-K` are allowed), then looks up the CIK. Each failure returns a clear error dict.

```

submissions = httpx.get(_SUBMISSIONS_URL.format(cik=cik), headers=headers, timeout=...)
submissions.raise_for_status()
recent = submissions.json().get("filings", {}).get("recent", {})
forms = recent.get("form", [])
accessions = recent.get("accessionNumber", [])
primary_docs = recent.get("primaryDocument", [])
dates = recent.get("filingDate", [])
for i, filed_form in enumerate(forms):
    if filed_form != form:
        continue
    accession = accessions[i].replace("-", "")
    filing_url =
f"https://www.sec.gov/Archives/edgar/data/{int(cik)}/{accession}/{primary_docs[i]}"
    doc = httpx.get(filing_url, headers=headers, timeout=...)
    doc.raise_for_status()
    return {"ticker": symbol, "form_type": form, "filing_date": dates[i],
            "filing_url": filing_url, "text": doc.text[:_MAX_TEXT_CHARS],
            "truncated": len(doc.text) > _MAX_TEXT_CHARS}
return {"error": f"No {form} filing found for {symbol}"}

```

- The SEC returns recent filings as several parallel lists (form types, accession numbers, document names, dates), all aligned by index. The loop scans for the first entry matching the requested form, builds the document URL from its parts, downloads it, and returns the text truncated to a maximum length with a `truncated` flag. If none match, it returns an error. Network errors are caught and returned as errors in the surrounding code.

3.10 `mcp_server/tools/rag.py`

Exposes retrieval as a tool with lazy setup.

```

_pipeline: RAGPipeline | None = None

def _get_pipeline() -> RAGPipeline:
    global _pipeline
    if _pipeline is None:
        _pipeline = RAGPipeline()
    return _pipeline

```

- The pipeline is created only on first use (lazy initialisation), so simply importing this module does not open the database or hit the network. `global` lets the function assign to the module level variable.

```
def search_earnings_documents(query, ticker=None, top_k=5) -> dict[str, Any]:
    if not query or not query.strip():
        return {"error": "query is required"}
    try:
        pipeline = _get_pipeline()
        chunks = pipeline.retrieve(query, top_k=top_k, ticker=ticker)
    except Exception as exc:
        return {"error": f"Retrieval failed: {exc}"}
    return {"query": query, "ticker": ticker.upper() if ticker else None,
            "count": len(chunks), "context": pipeline.format_context(chunks),
            "passages": [{"text": c.text, "citation": c.citation, "score": c.score} for c in
                           chunks]}
```

- It validates the query, retrieves passages, and returns both a ready to read `context` block and a structured `passages` list with citations and scores, so the agent can either quote the block or reason over individual passages.

3.11 `mcp_server/server.py`

The MCP server, a thin adapter over the shared tools.

```
from mcp.server.fastmcp import FastMCP
from mcp_server import tools
mcp = FastMCP("stock-intelligence")

@mcp.tool()
def get_financials(ticker: str) -> dict[str, Any]:
    """Fetch key fundamentals ... for a ticker.

    Args:
        ticker: Stock ticker symbol, e.g. AAPL, MSFT, TSLA.
    """
    return tools.get_financials(ticker)
```

- `FastMCP("stock-intelligence")` creates the server. `@mcp.tool()` registers the function as a callable MCP tool. The docstring is not decoration here: MCP sends it to the AI client so the model knows when and how to use the tool. Each wrapper simply calls the matching shared function, keeping logic out of the protocol layer. There are six such wrappers, one per tool.

```
def main() -> None:
    log.info("Starting stock-intelligence MCP server (stdio transport)")
    mcp.run()

if __name__ == "__main__":
    main()
```

- `mcp.run()` starts listening over standard input and output. The `if __name__ == "__main__"` guard means `main()` runs only when the file is executed directly (`python -m mcp_server.server`), not when it is imported. This is a standard Python idiom for "script entry point".

3.12 `agents/llm.py`

Builds the chat model from configuration.

```
def get_llm(model=None, temperature=None, max_tokens=None) -> ChatGroq:
    settings = get_settings()
    resolved_model = model or settings.groq_model
    return ChatGroq(model=resolved_model, api_key=settings.groq_api_key,
        temperature=settings.llm_temperature if temperature is None else temperature,
        max_tokens=max_tokens or settings.llm_max_tokens,
        timeout=settings.llm_request_timeout, max_retries=2)
```

- One place that constructs the `ChatGroq` client, so model, temperature, token limit and timeout all come from settings. Arguments allow overrides. Temperature again uses `is not None` so a caller can pass `0`. `max_retries=2` lets the client retry transient failures.

3.13 `agents/tools.py`

Wraps the shared functions as LangChain tools.

```
from langchain_core.tools import tool
from mcp_server import tools as impl

@tool
def get_financials(ticker: str) -> dict[str, Any]:
    """Get fundamental metrics ... Use for questions about profitability, size
    or financial health."""
    return impl.get_financials(ticker)
```

- `@tool` converts a function into a LangChain tool. The name, the type hints and the docstring become the tool's schema, which the model reads to decide when to call it. The docstrings

here are written for the model, describing when to use each tool. Every wrapper calls the same shared implementation (`impl`), so the agent and the MCP server share one code base.

```
def build_agent_tools() -> list:
    return [get_financials, get_price_history, calculate_ratios,
            get_analyst_recommendations, fetch_sec_filing, search_earnings_documents]
```

- Returns the full toolset the agent is allowed to use.

3.14 `agents/state.py`

Holds the system prompt.

```
SYSTEM_PROMPT = """You are a rigorous equity research analyst. ...
- Always fetch data with tools before making a claim; never rely on memory ...
- Prefer search_earnings_documents for qualitative questions ...
- When a tool returns an "error" field, acknowledge it and try an alternative ...
- End with a short, clearly-labelled "Bottom line" ...
You are an analytical assistant, not a financial adviser; do not tell the user to
buy or sell."""
```

- The system prompt is the standing instruction given to the model on every turn. It enforces the behaviour that matters: use tools for facts, cite sources, handle tool errors gracefully, summarise clearly, and avoid giving investment advice. Prompt design is part of the engineering, not an afterthought.

3.15 `agents/graph.py`

Builds and drives the ReAct agent.

```
def build_agent(model=None):
    llm = get_llm(model=model)
    tools = build_agent_tools()
    return create_react_agent(llm, tools, prompt=SYSTEM_PROMPT, checkpoint=MemorySaver())
```

- `create_react_agent` is LangGraph's prebuilt ReAct loop. Given a model, a list of tools and a prompt, it returns a compiled graph that will reason, call tools, read results and repeat. `MemorySaver()` is an in memory store of conversation state, keyed by a thread id, so a follow up question can continue an earlier conversation.

```
@dataclass
class EarningsAgent:
    model: str | None = None
    _graph: Any = field(default=None, init=False, repr=False)

    def __post_init__(self) -> None:
        self._graph = build_agent(self.model)
```

- A convenience wrapper. `_graph` is not a constructor argument (`init=False`) and is hidden from the printed form (`repr=False`); it is built in `__post_init__` , which a dataclass runs automatically right after construction. So creating `EarningsAgent()` compiles the graph once.

```
def analyze(self, question, thread_id="default", recursion_limit=25) -> str:
    result = self._graph.invoke(
        {"messages": [HumanMessage(content=question)]},
        config={"configurable": {"thread_id": thread_id}, "recursion_limit":
recursion_limit})
    messages = result.get("messages", [])
    for message in reversed(messages):
        if isinstance(message, AIMessage) and message.content:
            return message.content
    return "The agent did not produce a textual answer."
```

- `invoke` runs the whole loop to completion. The input is a list of messages starting with the user's question wrapped as a `HumanMessage` . `thread_id` selects the conversation; `recursion_limit` caps how many reason and act steps run before stopping, a guard against loops.
- The result contains the full message history. The code scans it backwards (`reversed`) for the last real answer from the model (`isinstance(message, AIMessage) and message.content`), skipping tool call bookkeeping messages.

```
def stream(self, question, thread_id="default"):
    yield from self._graph.stream(
        {"messages": [HumanMessage(content=question)]},
        config={"configurable": {"thread_id": thread_id}, "stream_mode="updates")
```

- The streaming variant yields each step as it happens, which a live UI can display. `yield from` re-emits every item from the graph's own stream.

3.16 app/main.py

The Streamlit web application. Streamlit reruns this whole script top to bottom on every interaction, which shapes how it is written.

```
import sys
from pathlib import Path
import streamlit as st
sys.path.insert(0, str(Path(__file__).resolve().parent.parent))
from core import get_settings
from mcp_server import tools
st.set_page_config(page_title="Stock Earnings Agent", page_icon="...", layout="wide")
```

- `sys.path.insert(0, project_root)` ensures the project packages are importable when Streamlit runs this file directly. `st.set_page_config` sets the browser tab title and page layout and must be the first Streamlit call.

```
@st.cache_resource(show_spinner=False)
def _load_agent():
    from agents import EarningsAgent
    return EarningsAgent()
```

- `@st.cache_resource` builds the agent once and reuses it across reruns and users, instead of recompiling the graph on every click. The import is inside the function so the heavy agent modules load only when first needed.

```
def _snapshot_tab():
    ticker = st.text_input("Ticker", value="AAPL", key="snap_ticker").strip().upper()
    period = st.selectbox("Price period", ["3mo", "6mo", "1y", "2y", "5y"], index=2)
    if not st.button("Fetch snapshot", type="primary"):
        return
    with st.spinner(f"Fetching data for {ticker}..."):
        financials = tools.get_financials(ticker)
        prices = tools.get_price_history(ticker, period)
        ratios = tools.calculate_ratios(ticker)
        analyst = tools.get_analyst_recommendations(ticker)
    if "error" in financials:
        st.error(financials["error"]); return
    c1, c2, c3, c4 = st.columns(4)
    c1.metric("Market cap", _human(financials.get("market_cap")))
    ...
```

- Each widget (`text_input` , `selectbox` , `button`) both draws the control and returns its current value. The early `return` after the button is the Streamlit pattern for "only do the work when clicked": on a normal rerun the button returns `False` . `st.spinner` shows a progress indicator during the calls. The snapshot calls the shared tools directly (no language model needed) and lays out results in columns and metrics. The small helpers `_human` , `_pct` , `_fmt` format numbers (for example trillions as `T`), returning a dash for missing values.

```
def _agent_tab():
    if "history" not in st.session_state:
        st.session_state.history = []
    for role, content in st.session_state.history:
        with st.chat_message(role):
            st.markdown(content)
    prompt = st.chat_input("...")
    if not prompt:
        return
    ...
    answer = _load_agent().analyze(prompt, thread_id="streamlit")
```

- `st.session_state` persists data across reruns for one browser session, here the chat history. Because the script reruns each time, the history is replayed from state on every run. `st.chat_input` returns the new message or `None` . The answer comes from the cached agent.

The ingest tab writes uploaded files to a temporary directory and calls `pipeline.ingest_paths` , then reports how many chunks were added. The `main` function checks configuration, and if a key is missing it shows the error and calls `st.stop()` so the rest of the page does not run.

3.17 evaluation/

`dataset.py` defines `EvalSample` (a question, a reference answer, the snippets a correct retrieval must contain, and an optional ticker) and a small default dataset.

`ragas_eval.py` has two evaluators:

```

class RetrievalEvaluator:
    def evaluate(self, dataset=None) -> RetrievalReport:
        data = list(dataset or default_dataset())
        hits = 0; reciprocal_ranks = 0.0
        for sample in data:
            chunks = self._pipeline.retrieve(sample.question, top_k=self._top_k,
            ticker=sample.ticker)
            rank = self._first_relevant_rank(chunks, sample.relevant_snippets)
            if rank is not None:
                hits += 1
                reciprocal_ranks += 1.0 / rank
        n = len(data)
        return RetrievalReport(samples=n, hit_rate=hits / n, mrr=reciprocal_ranks / n)

```

- This measures retrieval quality without using any model tokens. **Hit rate** is the fraction of questions where a correct passage appeared at all. **MRR** (mean reciprocal rank) rewards putting the correct passage higher: if the right passage is first the score is 1, if second it is 1/2, and so on. `_first_relevant_rank` returns the position of the first passage containing all required snippets.
- `GeminiEmbeddingsAdapter` wraps the embedding client in LangChain's expected interface so RAGAS can reuse it. `run_ragas_evaluation` generates an answer per question and scores faithfulness, answer relevancy and context precision and recall, returning a clear error dict if the model quota is exceeded rather than crashing.

3.18 `scripts/`

Small command line entry points. `healthcheck.py` verifies configuration, embeddings, the vector store and the model are all reachable, printing an `[OK]` or `[FAIL]` line for each and exiting non zero on failure, which suits a container readiness probe. `ingest.py` uses `argparse` to accept file paths and options and feeds them to the pipeline. `seed_eval_corpus.py` inserts a small demo document so the evaluation harness runs out of the box. Each ends with the `if __name__ == "__main__": main()` guard.

3.19 `tests/`

`conftest.py` holds shared fixtures. A fixture is a reusable setup function that pytest injects into tests by name.

```
@pytest.fixture(autouse=True)
def _dummy_env(monkeypatch):
    monkeypatch.setenv("GROQ_API_KEY", "test-groq-key")
    monkeypatch.setenv("GOOGLE_API_KEY", "test-google-key")
    from core.config import get_settings
    get_settings.cache_clear()
    yield
    get_settings.cache_clear()
```

- `autouse=True` applies it to every test automatically. `monkeypatch` safely sets environment variables and undoes the change after the test. `get_settings.cache_clear()` resets the cached settings so each test starts clean. The code before `yield` is setup; the code after is teardown.
- `FakeEmbeddingClient` produces small deterministic vectors offline, so tests never call the network. The `chroma_dir` and `collection_name` fixtures give the vector store tests one shared database path with unique collections, which avoids the native client conflict described earlier.

The test files use plain `assert` statements and `monkeypatch` to replace network calls with fakes. For example, `test_financials.py` swaps `yf.Ticker` for a fake object so it can test the mapping logic without hitting the market. `pytest.raises` asserts that a block raises a specific exception. The vector store tests are marked `integration` so they run in a separate process.

3.20 Project configuration files

- `pyproject.toml` is the single project descriptor: metadata, dependencies, the console script entry points, and settings for pytest and Ruff. The pytest section excludes integration tests by default and defines the `integration` marker.
- `requirements.txt` pins runtime dependencies for `pip install`. It documents why transformer libraries are avoided.
- `Dockerfile` builds the container: install dependencies in a cached layer, copy the code, create a non root user, expose the web port, and define a health check that runs the health check script.
- `Makefile` provides short commands (`make test`, `make run`, `make healthcheck` and so on) that wrap the longer underlying commands.

Part 4. End to end request traces

4.1 Snapshot request (no language model)

1. The user enters a ticker in the web app and clicks Fetch snapshot.
2. `app/main.py` calls the four shared tools in `mcp_server/tools/financials.py`.

3. Each tool validates the ticker, calls the market data library, and returns a clean dictionary, or an error dictionary.
4. The app formats the numbers and renders metrics, columns and tables.

4.2 Ask the analyst (full agent loop)

1. The user asks a question in the web app.
2. `EarningsAgent.analyze` wraps it as a message and calls the compiled graph.
3. The model reads the system prompt and the question and decides which tool to call first.
4. The chosen LangChain tool in `agents/tools.py` runs the same shared function the MCP server uses.
5. The tool result is returned to the model as context (the observation).
6. Steps 3 to 5 repeat until the model can answer, or the recursion limit is hit.
7. `analyze` scans the message history backwards for the final answer and returns it. The app displays it.

4.3 Document ingestion and retrieval

1. A file is uploaded in the app or passed to `python -m scripts.ingest`.
2. `ingestion.py` reads it into `Document` objects with metadata.
3. `chunking.py` splits them into overlapping chunks.
4. `embeddings.py` turns each chunk into a vector via the Gemini REST API.
5. `vector_store.py` upserts the vectors with stable ids into ChromaDB.
6. Later, `search_earnings_documents` embeds a query, ChromaDB returns the nearest chunks by cosine similarity, and the agent uses them as cited context.

4.4 MCP path

An MCP client (such as Claude) connects to `python -m mcp_server.server`, reads the six tool descriptions, and calls them directly. The same shared functions run, so the behaviour matches the agent and the web app exactly.

Part 5. Likely interview questions

Why separate the tools from the MCP server and the agent? So the business logic is written and tested once. The MCP server and the agent are thin adapters that reuse it, which prevents drift and duplication.

Why return errors as data instead of raising? Both an AI model and a UI need to reason about failure. A structured `{"error": ...}` value lets the caller decide what to do next, and it prevents one bad ticker from crashing a request.

Why compute embeddings with a REST call instead of a local model? To avoid heavy native machine learning libraries that are incompatible with the target platform, and to keep the deployment small. The trade off is a network dependency, which is handled with retries.

Why cache the ChromaDB client per path? The native core can crash if more than one client is opened for the same database path in a process. Caching guarantees a single shared client and also improves performance.

What is the ReAct loop and where is it in the code? It is the reason and act cycle the agent runs: think, call a tool, observe, repeat, then answer. It is created by `create_react_agent` in `agents/graph.py`.

How is retrieval quality measured? With hit rate and mean reciprocal rank in `RetrievalEvaluator`, which need no model tokens, plus RAGAS generation metrics when a budget is available.

Why do tests run in two groups? The vector store's native core conflicts with the data frame libraries in one process on Windows, so those tests are marked as integration and run separately.

How does configuration fail safely? All settings are validated by Pydantic at startup through `get_settings`. A missing key or invalid value raises immediately with a clear message rather than failing deep inside a request.

Why do logs go to standard error? Because the MCP protocol uses standard output. Any log written there would corrupt the protocol stream, so all logging is sent to standard error.